



**M.Kumarasamy  
College of Engineering**

Autonomous | Affiliated to Anna University Chennai

Approved by AICTE, Accredited by NBA and NAAC

Thalavapalayam, Karur - 639 113, Tamilnadu, India.



A Project Report

on

## **SHORTEST PATH FINDER**

Submitted in partial fulfilment of requirements for the award of the course

of

### **CGB1122 – DATA STRUCTURES**

Under the guidance of

**Mrs.S.PRABAVATHI M.E.,**

**Assistant Professor/CSE**

Submitted By

**PRIYADHARSHINI K (927622BCS128)**

**KARTHIKA S (927622BCS068)**

**PIRAKEERTHI R.S (927622BIT081)**

**DEPARTMENT OF FRESHMAN ENGINEERING**

**M.KUMARASAMY COLLEGE OF ENGINEERING**

(Autonomous)

**KARUR – 639 113**

**MAY 2026**



**M.Kumarasamy  
College of Engineering**

Autonomous | Affiliated to Anna University Chennai

Approved by AICTE, Accredited by NBA and NAAC  
Thalavapalayam, Karur - 639 113, Tamilnadu, India.



A Project Report

on

## **SHORTEST PATH FINDER**

Submitted in partial fulfilment of requirements for the award of the course

of

### **CGB1122 – DATA STRUCTURES**

Under the guidance of

**Mrs.S.PRABAVATHI M.E.,**

**Assistant Professor/CSE**

Submitted By

**PRIYADHARSHINI K (927622BCS128)**

**KARTHIKA S (927622BCS068)**

**PIRAKEERTHI R.S (927622BIT081)**

**DEPARTMENT OF FRESHMAN ENGINEERING**

**M.KUMARASAMY COLLEGE OF ENGINEERING**

**(Autonomous)**

**KARUR – 639 113**

**MAY 2026**



**M. Kumarasamy  
College of Engineering**

Autonomous | Affiliated to Anna University Chennai

Approved by AICTE, Accredited by NBA and NAAC

Thalavapalayam, Karur - 639 113, Tamilnadu, India.



## **M. KUMARASAMY COLLEGE OF ENGINEERING**

**(Autonomous Institution affiliated to Anna University, Chennai)**

**KARUR – 639 113**

### **BONAFIDE CERTIFICATE**

Certified that this project report on“ **SHORTEST PATH FINDER**” is the bonafide work of **PRIYADHARSHINI K(927625BCS128),KARTHIKA S(927625BCS068) PIRAKEERTHI R.S(927625BIT081)**Who carried out the project work during the academic year 2025 - 2026 under my supervision

Signature

**Mrs.S.PRABAVATHI,M.E.,**

**SUPERVISOR,**

Department of Computer Science and Engineering,

M. Kumarasamy College of Engineering,

Thalavapalayam, Karur -639 113.

Signature

**Dr. K.CHITIRAKALA, M.Sc., M.Phil.,Ph.D.,**

**HEAD OF THE DEPARTMENT,**

Department of Freshman Engineering,

M. Kumarasamy College of Engineering,

Thalavapalayam, Karur -639 113.

Submitted for the End Semester Review held on \_\_\_\_\_

**INTERNAL EXAMINER**



**M.Kumarasamy  
College of Engineering**

Autonomous | Affiliated to Anna University Chennai  
Approved by AICTE, Accredited by NBA and NAAC  
Thalavapalayam, Karur - 639 113, Tamilnadu, India.



## **VISION OF THE INSTITUTION**

To emerge as a leader among the top institutions in the field of technical education

## **MISSION OF THE INSTITUTION**

- Produce smart technocrats with empirical knowledge who can surmount the global challenges
- Create a diverse, fully-engaged, learner-centric campus environment to provide quality education to the students
- Maintain mutually beneficial partnerships with our alumni, industry, and Professional associations

## **PROGRAM OUTCOMES (POs)**

Engineering students will be able to:

**PO1: Engineering knowledge:** Apply knowledge of mathematics, natural science computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.

**PO2: Problem analysis: :** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development

**PO3: Design/development of solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions

**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective



**M.Kumarasamy  
College of Engineering**

Autonomous | Affiliated to Anna University Chennai

Approved by AICTE, Accredited by NBA and NAAC  
Thalavapalayam, Karur - 639 113, Tamilnadu, India.



reports and design documentation, make effective presentations

**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to

one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments

**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change



**M. Kumarasamy  
College of Engineering**

Autonomous | Affiliated to Anna University Chennai  
Approved by AICTE, Accredited by NBA and NAAC  
Thalavapalayam, Karur - 639 113, Tamilnadu, India.



## **ABSTRACT**

The Shortest Path Finder project focuses on finding the minimum distance between two vertices in a graph using data structures. In this project, a graph is used to represent nodes and edges, where edges may contain weights indicating distance or cost. The main objective is to determine the shortest and most efficient path from a source node to a destination node. This problem is solved using graph-based algorithms and data structures such as adjacency lists, arrays, and priority queues. The project helps in understanding how graphs are applied in solving real-world problems like transportation systems, network routing, and navigation. Overall, this project demonstrates the practical use of data structures in optimizing path selection. The primary objective of the project is to determine the shortest and most efficient route from a given source node to a destination node. To achieve this, the project utilizes graph algorithms such as Dijkstra's Algorithm, Breadth-First Search (BFS) for unweighted graphs, and other shortest path techniques depending on the graph structure. The graph is represented using efficient data structures such as adjacency lists, arrays, and priority queues, which help reduce memory usage and improve performance when processing datasets.



## ABSTRACT WITH POs MAPPING

| ABSTRACT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | POs<br>MAPPED                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>The Shortest Path Finder project focuses on finding the minimum distance between two vertices in a graph using data structures. In this project, a graph is used to represent nodes and edges, where edges may contain weights indicating distance or cost. The main objective is to determine the shortest and most efficient path from a source node to a destination node. This problem is solved using graph-based algorithms and data structures such as adjacency lists, arrays, and priority queues.</p> <p>The project helps in understanding how graphs are applied in solving real-world problems like transportation systems, network routing, and navigation. Overall, this project demonstrates the practical use of data structures in optimizing path selection.</p> <p>The primary objective of the project is to determine the shortest and most efficient route from a given source node to a destination node. To achieve this, the project utilizes graph algorithms such as Dijkstra's Algorithm, Breadth-First Search (BFS) for unweighted graphs, and other shortest path techniques depending on the graph structure. The graph is represented using efficient data structures such as adjacency lists, arrays, and priority queues, which help reduce memory usage and improve performance when processing datasets.</p> | <p><b>PO1(2)</b></p> <p><b>PO2(3)</b></p> <p><b>PO3(2)</b></p> <p><b>PO4(2)</b></p> <p><b>PO5(3)</b></p> <p><b>PO9(3)</b></p> <p><b>PO10(3)</b></p> <p><b>PO11(2)</b></p> <p><b>PO12(2)</b></p> |

Note:1-LOW,2-Medium,3-High

**SUPERVISOR**

**HEAD OF THE DEPARTMENT**





## TABLE OF CONTENTS

| CHAPTER<br>No. | TITLE                            | PAGE<br>No. |
|----------------|----------------------------------|-------------|
| <b>1</b>       | <b>Introduction</b>              | <b>1</b>    |
|                | 1.1 Introduction                 | <b>1</b>    |
|                | 1.2 Objective                    | <b>1</b>    |
|                | 1.3 Data Structure Choice        | <b>1</b>    |
| <b>2</b>       | <b>Project Methodology</b>       | <b>2</b>    |
|                | 2.1 Shortest Path Finder         | <b>2</b>    |
|                | 2.2 Block Diagram                | <b>3</b>    |
| <b>3</b>       | <b>Modules</b>                   | <b>4</b>    |
|                | 3.1 Create Graph Module          | <b>4</b>    |
|                | 3.2 Display Graph Module         | <b>4</b>    |
|                | 3.3 Find Shortest Path Module    | <b>4</b>    |
|                | 3.4 Display Shortest Path Module | <b>4</b>    |
|                | 3.5 Update Graph Module          | <b>4</b>    |
| <b>4</b>       | <b>Results and Discussion</b>    | <b>5</b>    |
| <b>5</b>       | <b>Conclusion</b>                | <b>8</b>    |
|                | <b>References</b>                | <b>8</b>    |
|                | <b>Appendix</b>                  | <b>9</b>    |

# CHAPTER 1

## INTRODUCTION

### 1.1 Introduction

Shortest Path Finder is a basic concept in Data Structures used to find the minimum distance between two points in a graph. In this project, a graph data structure is used where nodes represent different locations and edges represent the connection between them. Each connection may have a weight that shows the distance or cost.

The main aim of this project is to find the shortest and most efficient path from a starting node to a destination node. This problem is solved using graph-based methods in data structures. It helps in understanding how graphs are used to solve real-world problems like maps and network routing.

### 1.2 Objective

The main objective of this project is to design and develop a shortest path finder system that determines the minimum distance between nodes in a graph. It uses efficient algorithms like Dijkstra's Algorithm for accurate graph traversal. The system provides a user-friendly interface and displays both the shortest path and total cost. It also supports dynamic graphs and helps understand real-world problem solving using data structures and algorithms.

### 1.3 Data Structure Choice

The choice of data structure plays a crucial role in designing an efficient shortest path finder system. In this project, the graph is represented using an adjacency list, which stores nodes and their connected edges in a compact form. This approach is memory-efficient compared to adjacency matrices, especially for sparse graphs. Each node maintains a list of its neighboring nodes along with edge weights.

To efficiently determine the next node with the smallest distance, a priority queue (min-heap) is used. This allows the algorithm to always process the closest unvisited node first. The use of a priority queue significantly improves the time complexity of Dijkstra's Algorithm. Arrays or dictionaries are used to store distances and track visited nodes during execution.

## **CHAPTER 2**

### **PROJECT METHODOLOGY**

#### **2.1 Shortest Path Finder**

A shortest path finder is a system designed to determine the minimum distance or cost between nodes in a graph. It plays a crucial role in solving real-world problems such as route optimization, network routing, and navigation systems. The project uses graph data structures where nodes represent locations and edges represent connections with associated weights (distance or cost).

In this project, Dijkstra's Algorithm is used to compute the shortest path from a source node to a destination node. The algorithm works by selecting the node with the smallest tentative distance, updating its neighboring nodes, and repeating the process until the shortest path to all nodes is determined.

Key features of the shortest path finder system:

##### **1.Graph Representation:**

The system represents data using a graph structure consisting of vertices (nodes) and edges (connections). Each edge has a weight that indicates the cost or distance between nodes.

##### **2.Efficient Path Calculation:**

Dijkstra's Algorithm ensures efficient computation of the shortest path by systematically selecting the minimum distance node and updating adjacent nodes.

##### **3.Dynamic Input Handling:**

Users can input different nodes, edges, and weights. The system can handle dynamic graphs with varying sizes and complexities.

##### **4.Accurate Output Display:**

The system displays the shortest path along with the total distance or cost. It clearly shows the sequence of nodes from source to destination.

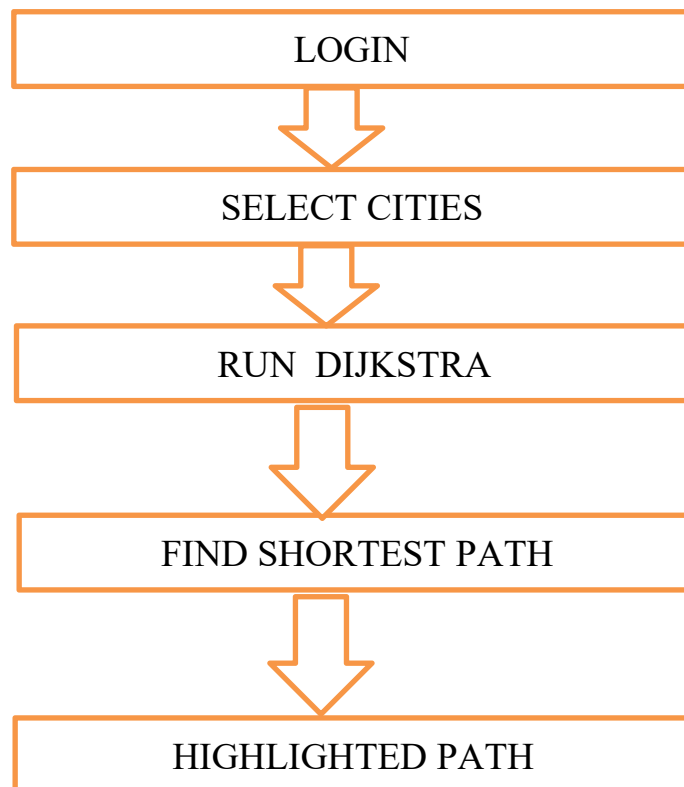
### **5.Scalability and Flexibility:**

The algorithm can be applied to both small and large graphs, making it suitable for real-world applications like GPS navigation, network routing, and logistics planning.

### **6.Directed and Undirected Graph Support:**

The system can be extended to support both directed and undirected graphs, depending on the nature of the problem.

## **2.2 Block Diagram**



## **CHAPTER 3**

### **MODULES**

#### **3.1 Create Graph**

This module enables the user to construct the graph by entering nodes and edges. The user is prompted to provide the number of vertices and the connections between them along with their respective weights. A suitable data structure such as an adjacency list or adjacency matrix is initialized to store the graph. Memory is allocated dynamically, and all nodes and edges are stored efficiently for further operations.

#### **3.2 Display Graph**

This module displays the complete structure of the graph. It traverses the graph representation and shows each node along with its connected nodes and corresponding edge weights. This helps the user verify the correctness of the graph input before performing shortest path calculations.

#### **3.3 Find Shortest Path**

This is the core module of the system. It uses Dijkstra's Algorithm to calculate the shortest path from a selected source node to a destination node. The algorithm initializes distances, updates them iteratively, and selects the node with the minimum distance at each step. It ensures efficient computation of the shortest path in a weighted graph without negative edges.

#### **3.4 Display Shortest Path**

After computing the shortest path, this module presents the result to the user. It displays the sequence of nodes involved in the shortest path along with the total distance or cost. The path is reconstructed using predecessor or parent tracking from the algorithm.

#### **3.5 Update Graph (Optional)**

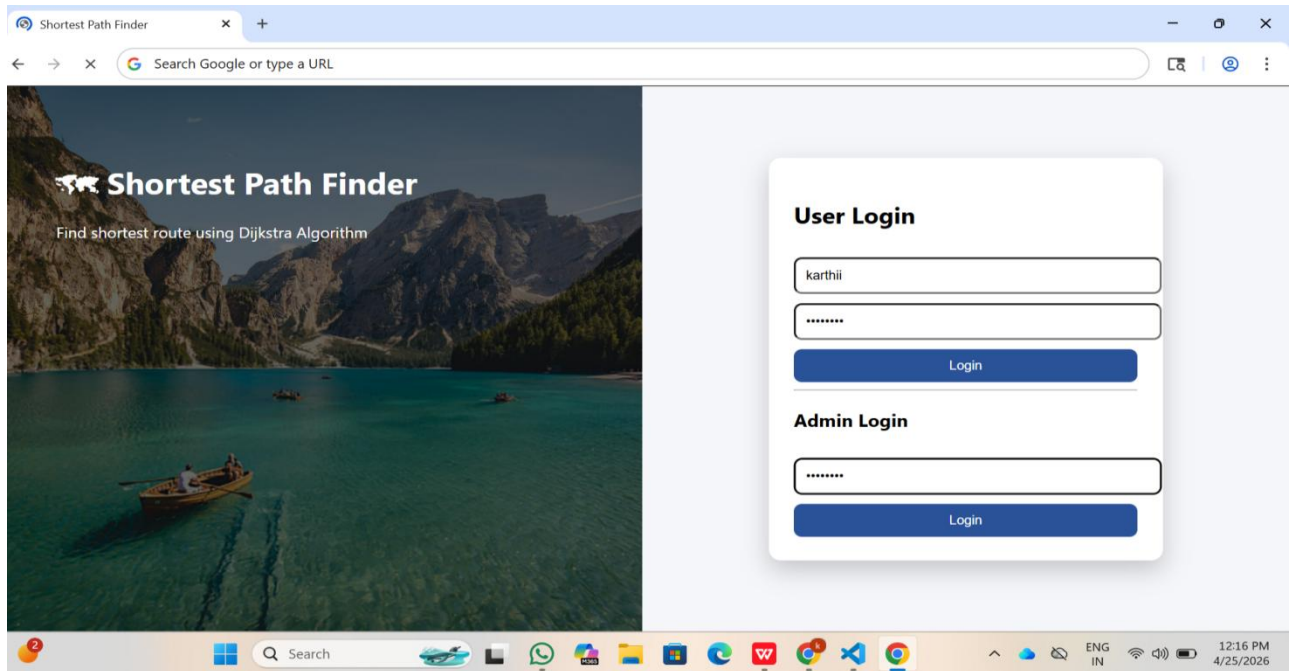
This module allows the user to modify the existing graph dynamically. Users can add or remove nodes and edges or update edge weights. The system updates the graph structure accordingly and ensures consistency. The modified graph can then be reused for further shortest path calculations without restarting the program.

# CHAPTER 4

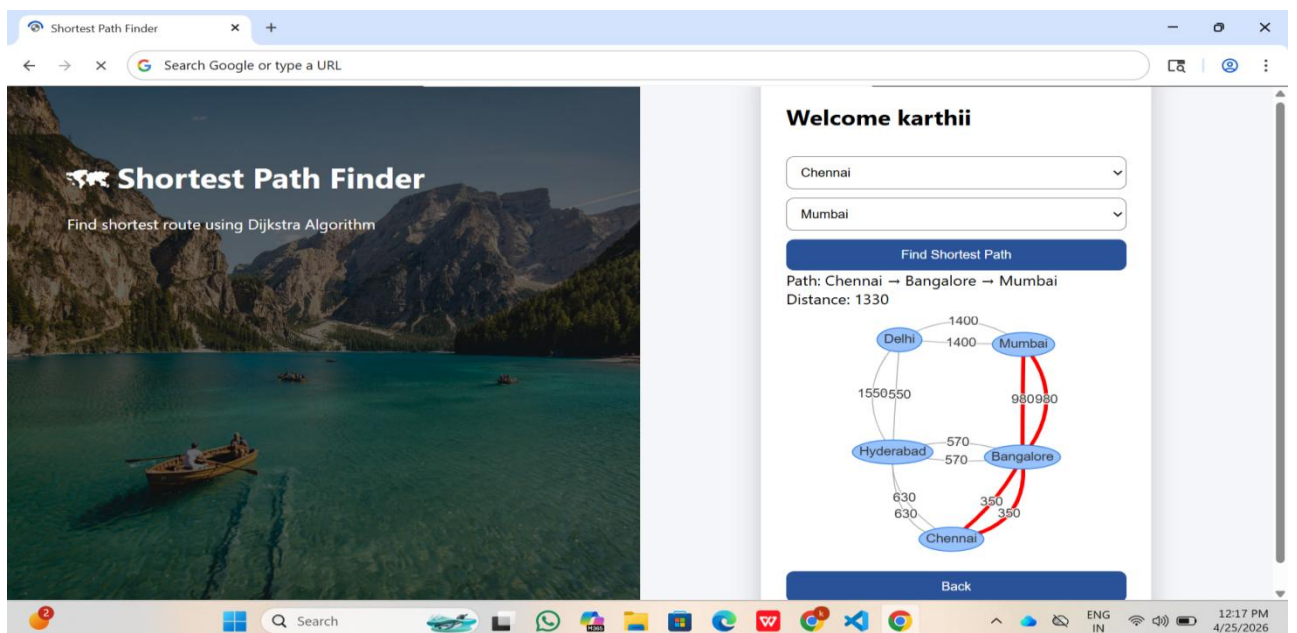
## RESULTS AND DISCUSSION

### 4.1 Results

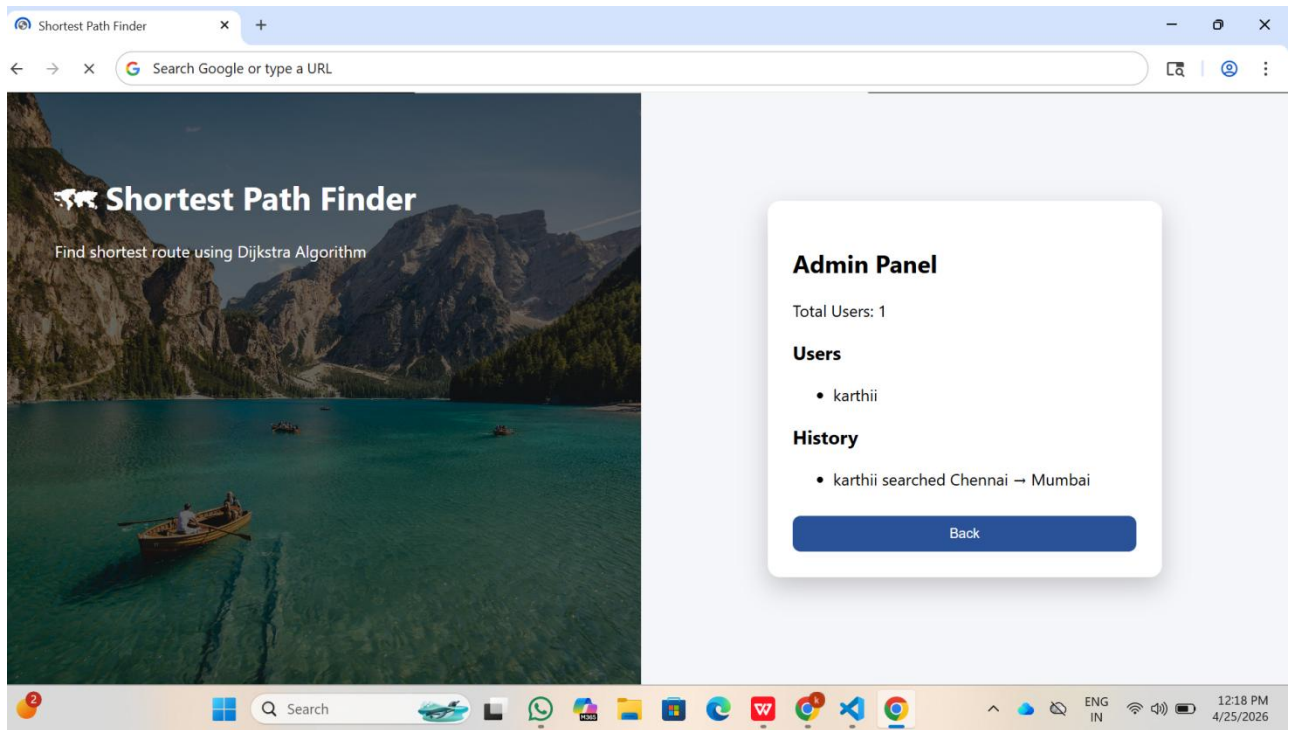
#### 4.1.1 USER AND ADMIN LOGIN PAGE



#### 4.1.2 GRAPH-BASED ROUTE OPTIMIZATION INTERFRAN



### 4.1.3 ADMIN DASHBOARD OF SHORTEST PATH FINDER



## 4.2 Discussion

The Shortest Path Finder project demonstrates how graph-based problems can be solved efficiently using data structures and algorithms. The system models real-world scenarios such as road networks, communication systems, and navigation maps, where finding the minimum distance between two points is essential.

In this project, a graph is used to represent nodes (locations) and edges (paths) with weights (distance or cost). Dijkstra's Algorithm is applied to compute the shortest path from a source node to all other nodes. The algorithm works by selecting the node with the smallest tentative distance and updating its neighboring nodes iteratively. This ensures that the shortest path is found in an optimal and systematic manner.

The use of data structures like adjacency lists or matrices improves storage efficiency and traversal speed. Priority queues can further optimize performance by reducing the time complexity of selecting the minimum distance node. The system is designed to handle dynamic inputs, allowing users to create and modify graphs easily.



## **CHAPTER 5**

### **CONCLUSION**

The Shortest Path Finder project successfully demonstrates how graph data structures and Dijkstra Algorithm can be used to find the minimum distance between nodes. The system efficiently calculates the shortest path and provides accurate results for weighted graphs.

This project helps in understanding the practical application of data structures in real-world problems such as navigation and network routing. Overall, the system is simple, efficient, and easy to implement, making it a useful solution for path optimization problems.

### **REFERENCES**

1. Thomas H. Cormen, Introduction to Algorithms, MIT Press
2. Mark Allen Weiss, Data Structures and Algorithm Analysis
3. GeeksforGeeks – Dijkstra Algorithm explanation
4. TutorialsPoint – Data Structures concepts
5. W3Schools – Basic programming reference

## APPENDIX

```
<!DOCTYPE html>
<html>
<head>
  <title>Shortest Path Finder</title>

  <script src="https://unpkg.com/vis-network/standalone/umd/vis-network.min.js"></script>

<style>
  body {
    margin: 0;
    font-family: 'Segoe UI';
    display: flex;
    height: 100vh;
  }

  .left {
    width: 50%;
    position: relative;
    background: url("https://images.unsplash.com/photo-1501785888041-af3ef285b470")
no-repeat center;
    background-size: cover;
    color: white;
  }

  .left::before {
    content: "";
    position: absolute;
    width: 100%;
```

```

    height: 100%;
    background: rgba(0,0,0,0.6);
}

.left-content {
    position: relative;
    z-index: 2;
    padding: 60px;
}

.right {
    width: 50%;
    display: flex;
    justify-content: center;
    align-items: center;
    background: #f5f6fa;
}

.box {
    background: white;
    padding: 25px;
    border-radius: 12px;
    width: 340px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.2);
}

input, select, button {
    width: 100%;
    padding: 10px;
    margin-top: 10px;
    border-radius: 8px;
}

```

```

    button {
        background: #2a5298;
        color: white;
        border: none;
        cursor: pointer;
    }

    #userPanel, #adminPanel { display: none; }

    #network {
        height: 280px;
        margin-top: 10px;
    }
</style>
</head>

<body>

<!-- LEFT -->
<div class="left">
    <div class="left-content">
        <h1> Shortest Path Finder</h1>
        <p>Find shortest route using Dijkstra Algorithm</p>
    </div>
</div>

<!-- RIGHT -->
<div class="right">

    <!-- HOME -->
    <div class="box" id="home">
        <h2>User Login</h2>
        <input id="username" placeholder="Enter Name">

```

```

<input type="password" id="userPass" placeholder="Password">
<button onclick="userLogin()">Login</button>

<hr>

<h3>Admin Login</h3>
<input type="password" id="adminPass" placeholder="Admin Password">
<button onclick="adminLogin()">Login</button>
</div>

<!-- USER -->
<div class="box" id="userPanel">
  <h2>Welcome <span id="userName"></span></h2>

  <select id="from"></select>
  <select id="to"></select>

  <button onclick="findRoute()">Find Shortest Path</button>

  <div id="result"></div>
  <div id="network"></div>

  <button onclick="goHome()">Back</button>
</div>

<!-- ADMIN -->
<div class="box" id="adminPanel">
  <h2>Admin Panel</h2>

  <p>Total Users: <span id="count"></span></p>

  <h3>Users</h3>
  <ul id="userList"></ul>

```

```

    <h3>History</h3>
    <ul id="historyList"></ul>

    <button onclick="goHome()">Back</button>
</div>

</div>

<script>
let graph = {
  Chennai: { Bangalore: 350, Hyderabad: 630 },
  Bangalore: { Chennai: 350, Hyderabad: 570, Mumbai: 980 },
  Hyderabad: { Chennai: 630, Bangalore: 570, Delhi: 1550 },
  Mumbai: { Bangalore: 980, Delhi: 1400 },
  Delhi: { Hyderabad: 1550, Mumbai: 1400 }
};

let users = [];
let history = [];
let currentUser = "";

const USER_PASSWORD = "user123";
const ADMIN_PASSWORD = "admin123";

let nodes = new vis.DataSet();
let edges = new vis.DataSet();
let network;

// LOGIN USER
function userLogin() {
  let name = username.value;
  let pass = userPass.value;

```

```

if (!name || !pass) return alert("Enter details");

if (pass !== USER_PASSWORD) return alert("Wrong password");

currentUser = name;
users.push(name);

home.style.display = "none";
userPanel.style.display = "block";

userName.innerText = name;

loadPlaces();
drawGraph();
}

// ADMIN
function adminLogin() {
  if (adminPass.value !== ADMIN_PASSWORD) {
    return alert("Wrong admin password");
  }

  home.style.display = "none";
  adminPanel.style.display = "block";
  updateAdmin();
}

// HOME
function goHome() {
  userPanel.style.display = "none";
  adminPanel.style.display = "none";
  home.style.display = "block";
}

```

```

}

// ADMIN DATA
function updateAdmin() {
  count.innerText = users.length;

  userList.innerHTML = users.map(u => `<li>${u}</li>`).join("");

  historyList.innerHTML = history.map(h => `<li>${h}</li>`).join("");
}

// PLACES
function loadPlaces() {
  let places = Object.keys(graph);

  from.innerHTML = "";
  to.innerHTML = "";

  places.forEach(p => {
    from.innerHTML += `<option>${p}</option>`;
    to.innerHTML += `<option>${p}</option>`;
  });
}

// GRAPH
function drawGraph() {
  nodes.clear();
  edges.clear();

  for (let n in graph) {
    nodes.add({ id: n, label: n });
  }
}

```



```

for (let a in graph) {
  for (let b in graph[a]) {
    edges.add({
      id: a + "-" + b,
      from: a,
      to: b,
      label: graph[a][b].toString(),
      color: "#aaa",
      width: 1
    });
  }
}

network = new vis.Network(
  document.getElementById("network"),
  { nodes, edges },
  {}
);
}

```

// DIJKSTRA

```

function dijkstra(start) {
  let dist = {}, parent = {}, visited = {};

  for (let n in graph) {
    dist[n] = Infinity;
    parent[n] = null;
    visited[n] = false;
  }

  dist[start] = 0;

  for (let i in graph) {

```

```

    let min = null;

    for (let n in dist) {
        if (!visited[n] && (min === null || dist[n] < dist[min])) {
            min = n;
        }
    }

    visited[min] = true;

    for (let neigh in graph[min]) {
        let newDist = dist[min] + graph[min][neigh];

        if (newDist < dist[neigh]) {
            dist[neigh] = newDist;
            parent[neigh] = min;
        }
    }
}

return { dist, parent };
}

// FIND PATH
function findRoute() {
    let start = from.value;
    let end = to.value;

    let { dist, parent } = dijkstra(start);

    let path = [];
    let cur = end;

```

```

while (cur) {
  path.unshift(cur);
  cur = parent[cur];
}
result.innerHTML =
  "Path: " + path.join(" → ") +
  "<br>Distance: " + dist[end];
// reset edges
edges.forEach(e => {
  edges.update({ id: e.id, color: "#aaa", width: 1 });
});
// highlight path
for (let i = 0; i < path.length - 1; i++) {
  let id1 = path[i] + "-" + path[i + 1];
  let id2 = path[i + 1] + "-" + path[i];
  if (edges.get(id1)) {
    edges.update({ id: id1, color: "red", width: 4 });
  }
  if (edges.get(id2)) {
    edges.update({ id: id2, color: "red", width: 4 });
  }
}
network.setData({ nodes, edges });
history.push(currentUser + " searched " + start + " → " + end);
}
</script>
</body>
</html>

```